

Evaluation Report for the Slovenian Tax RAG Assistant

Current Baseline, Retrieval Diagnostics, and Recommendations

ByteBanda NLP Course Project

29 April 2026

Abstract

This report documents the current version of the Slovenian tax Retrieval-Augmented Generation (RAG) assistant and summarizes an evaluation run performed on the ARNES HPC environment. The system ingests Slovenian tax legislation, chunks extracted text, builds a FAISS vector index using multilingual sentence embeddings, retrieves the top ranked chunks for a user question, and generates grounded answers with a local Mistral-style language model. The evaluation uses 20 manually written questions with expected sources, expected legal locations, expected chunk identifiers, key phrases, and reference answers. The current generation stack runs successfully without runtime failures, but retrieval quality is limited: source hit@3 is 40.0%, exact expected chunk hit@3 is 0.0%, and exact phrase coverage is 0.0%. A second manual answer assessment gives average scores of 0.70/2.00 for context relevance, 1.05/2.00 for faithfulness, and 0.20/2.00 for answer correctness. These results indicate that the present baseline is technically functional but retrieval is not yet reliable enough for legal question answering.

1 System Overview

The current system is a deliberately simple RAG baseline for Slovenian tax questions. The implementation is organized as a transparent Python pipeline rather than a larger framework-based application. Its main execution path is:

HTML tax documents → text extraction → character chunks → embeddings → FAISS retrieval → local LLM generation.

The important source modules are listed in Table 1.

Module	Role
<code>src/ingest.py</code>	Loads HTML, PDF, Markdown, and text files; extracts and normalizes visible text.
<code>src/chunking.py</code>	Splits documents into overlapping character-based chunks.
<code>src/build_index.py</code>	Embeds chunks and writes <code>data/index/faiss.index</code> and <code>data/index/chunks.jsonl</code> .
<code>src/retrieve.py</code>	Loads the FAISS index and returns top-k chunks for a question.
<code>src/generate_answer.py</code>	Builds the grounded prompt, loads the local LLM, generates an answer, and appends sources.
<code>src/evaluate_rag.py</code>	Runs the current evaluation suite and writes readable logs plus structured JSONL results.

Table 1: Main modules in the current RAG implementation.

1.1 Data

The evaluation uses seven Slovenian tax-law HTML files from PISRS:

- ZAK04701_zddv-1_NPB20.html: Value Added Tax Act (ZDDV-1).
- PRAV7542_pravilnik-zddv-1_NPB30.html: Rules related to ZDDV-1.
- ZAK04697_zdoh-2_NPB36.html: Personal Income Tax Act (ZDoh-2).
- ZAK04687_zddpo-2_NPB23.html: Corporate Income Tax Act (ZDDPO-2).
- PRAV8299_pravilnik-zddpo-2_NPB2.html: Rules related to ZDDPO-2.
- ZAK04703_zdavn-2_NPB31.html: Tax Procedure Act (ZDavnP-2).
- PRAV7927_pravilnik-zdavn-2_NPB25.html: Rules related to ZDavnP-2.

During the recorded HPC run, these files produced 1,897 text chunks from seven document records. The chunker uses a default chunk size of 1,200 characters and an overlap of 200 characters.

1.2 Embedding and Retrieval

The embedding model is `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`. Chunk embeddings are normalized, and FAISS `IndexFlatIP` is used for inner product search. Because embeddings are normalized, this is equivalent to cosine similarity ranking. The current default retrieval depth is `top_k=3`.

1.3 Technical Retrieval Mechanics

The retrieval component maps both documents and questions into the same vector space. During index construction, every chunk text c_i is encoded by the SentenceTransformers model:

$$\mathbf{v}_i = f(c_i)$$

The implementation then normalizes each embedding:

$$\hat{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2}.$$

At query time, the user question q is encoded and normalized in the same way:

$$\hat{\mathbf{u}} = \frac{f(q)}{\|f(q)\|_2}.$$

FAISS ranks chunks by inner product:

$$s_i = \hat{\mathbf{u}}^\top \hat{\mathbf{v}}_i.$$

Because both vectors are unit-normalized, this score is cosine similarity. A higher score means that the question and chunk are closer in the embedding space. The system returns the three chunks with the highest scores and passes their raw text, source filenames, chunk IDs, ranks, and scores to the generator.

This design is simple and reproducible, but it has an important limitation for legal text: dense semantic similarity often finds topically related passages rather than the exact statutory article. For example, a question about a ZDDV-1 definition can retrieve a nearby implementing-rule passage because it contains many of the same tax terms. The current implementation does not yet use article metadata, lexical matching, source-type boosting, or reranking.

1.4 Answer Generation

Answer generation uses the local model path:

```
/d/hpc/projects/onj_fri/models/intent
```

The prompt instructs the model to answer only from retrieved context, cite source filenames and chunk IDs, avoid definitive legal advice, and answer in the language of the question. The current tokenizer is loaded with `use_fast=False` to support the local Llama/Mistral tokenizer files, and the environment includes `sentencepiece` and `protobuf`. The default maximum generation length is 1,024 new tokens in the CLI, while the full evaluation job used 512 new tokens per question to keep runtime bounded.

2 Evaluation Design

The evaluation suite is stored in:

```
evaluation/tax_eval_questions.jsonl
```

It contains 20 manually constructed questions across four legal domains: VAT (DDV), personal income tax (ZDoh-2), corporate income tax (ZDDPO-2), and tax procedure (ZDavP-2). Each example contains:

- the user question,
- expected source file(s),
- expected chunk ID(s),
- expected legal location(s), such as article names,
- key phrases that should appear in retrieved context,
- a concise reference answer.

The HPC evaluation script is:

```
slurm/run_rag_eval.sh
```

It rebuilds the FAISS index and runs:

```
python -m src.evaluate_rag -questions evaluation/tax_eval_questions.jsonl -top-k  
3
```

The recorded run wrote human-readable results to `logs/tax-rag-eval-14619040.out` and structured results to `logs/tax-rag-eval-14619040.jsonl`.

3 Metrics

Three retrieval-oriented metrics are used:

Source hit@3 Whether at least one retrieved chunk comes from an expected source file.

Chunk hit@3 Whether at least one retrieved chunk exactly matches an expected chunk ID.

Expected phrase coverage Whether all manually specified key phrases are present in the retrieved context.

The generation step is also monitored for runtime failures. No automatic semantic answer grading is used in the pipeline itself. For this report, the generated answers were additionally reviewed manually with three 0–2 scores:

Context relevance 0 means the retrieved context is unrelated to the expected legal rule; 1 means it is topically related but misses the exact rule; 2 means it contains the expected legal rule.

Faithfulness 0 means the answer introduces unsupported claims; 1 means it mostly follows the retrieved context but overgeneralizes or misuses it; 2 means it is well grounded in the retrieved context.

Answer correctness 0 means the answer is wrong for the reference question; 1 means partially correct; 2 means correct.

4 Results

Table 2 summarizes the full 20-question run.

Metric	Count	Rate
Questions	20	–
Source hit@3	8 / 20	40.0%
Expected chunk hit@3	0 / 20	0.0%
All expected phrases found@3	0 / 20	0.0%
Generation failures	0 / 20	0.0%

Table 2: Overall evaluation results from `tax-rag-eval-14619040`.

Table 3 aggregates source-level retrieval by broad legal domain.

Domain	Questions	Source hits	Source hit rate
DDV / ZDDV-1	7	3	42.9%
ZDoh-2	5	4	80.0%
ZDDPO-2	5	0	0.0%
ZDavP-2	3	1	33.3%

Table 3: Source hit@3 by broad legal domain.

The exact chunk metric is intentionally strict. None of the 20 cases retrieved the manually expected chunk ID in the top three results. In several cases the retriever found the correct source document but an adjacent or procedurally related chunk. This is still insufficient for statutory question answering because the generator can only answer from the text that was actually passed in the prompt.

4.1 Manual Answer Scores

Table 4 summarizes the manual 0–2 scoring pass over the generated answers in `logs/tax-rag-eval-14619040.js`

Manual metric	Score sum	Mean	Normalized
Context relevance	14 / 40	0.70 / 2	35.0%
Faithfulness	21 / 40	1.05 / 2	52.5%
Answer correctness	4 / 40	0.20 / 2	10.0%

Table 4: Manual assessment of generated answers.

The distribution of manual scores is also informative. Context relevance was never scored 2 because no retrieved context contained the exact expected chunk. Fourteen cases received context relevance 1, meaning the retrieved text was tax-related but not the exact statutory rule. Faithfulness is higher than correctness because the model often stays close to the wrong retrieved context. Only four answers were partially correct, and none were fully correct under this strict reference-answer evaluation.

ID	Ctx.	Faith.	Corr.	Judgement
ddv_01	1	1	0	VAT-related rulebook chunks, but not the statutory object of VAT.
ddv_02	1	1	0	Confuses taxpayer definition with persons liable to pay VAT.
ddv_03	1	1	0	Uses partial-supply invoice context instead of the definition of supply of goods.
ddv_04	1	1	0	Retrieves a ZDDV-1 chunk, but not the service-definition article.
ddv_05	1	1	1	Near miss: special Article 33.a context instead of the general Article 33 rule.
ddv_06	0	2	0	Faithfully refuses because rates are absent from retrieved context.
ddv_07	1	1	1	Partially useful, but over-expanded with rulebook details.
doh_01	1	2	0	Correct statute, wrong chunk; model refuses exact definition.
doh_02	1	1	0	Uses dependent-family/resident-relief context, not taxpayer scope.
doh_03	0	1	0	Retrieved context is unrelated to source-of-income rule.
doh_04	1	1	0	Lists a narrow special case instead of Article 18 income categories.
doh_05	1	1	0	Describes special relief for new residents, not general relief.
ddpo_01	1	1	0	Uses procedural withholding context, not ZDDPO-2 taxpayer definition.
ddpo_02	0	0	0	Unsupported answer with unrelated context.
ddpo_03	1	1	1	Partially captures the tax-base idea, but imprecisely.
ddpo_04	0	2	0	Faithfully says the rate is missing from retrieved sources.
ddpo_05	0	0	0	Unrelated context and hallucinated description of ZDDPO-2.
zdavp_01	1	1	0	Rulebook context, not Article 11 list of tax authorities.
zdavp_02	1	1	1	Partially relevant taxpayer/payer terminology, but too narrow.
zdavp_03	0	1	0	Context is about security instruments, not right to information.

Table 5: Per-case manual scores: context relevance, faithfulness, and answer correctness.

The best domain-level retrieval performance is observed for ZDoh-2 questions, where four of five questions retrieved at least one chunk from the expected source document. The weakest area is ZDDPO-2, where none of the five questions retrieved the expected source file in the top three chunks.

5 Qualitative Analysis

5.1 Generation Works but Depends Strongly on Retrieved Context

The local LLM loaded successfully on the GPU and produced an answer for every question. This is an important engineering milestone: the container, dependencies, tokenizer configuration, GPU execution, and model loading now work end to end. However, the answer quality follows the quality of retrieved chunks. When the retrieved chunks are off-topic, the generated answer is also off-topic or explicitly says that the requested information is not available.

5.2 Example: Near Miss for DDV Obračun

For the question:

When does the taxable event and VAT accounting obligation arise for supplies of goods or services?

the expected legal location was ZDDV-1, Article 33, and the expected answer was: the event arises when goods are supplied or services are performed. The top retrieved source was still `ZAK04701_zddv-1_NPB20.html`, but it returned `chunk_000904` rather than the expected `chunk_000902`. The generated answer therefore focused on a narrower special case around Article 33.a instead of the general rule from Article 33. This illustrates that source-level retrieval can be correct while local chunk selection is still too imprecise for legal QA.

5.3 Example: Miss for Basic Dohodnina Definition

For the question:

What is dohodnina according to ZDoh-2?

the expected answer was the simple definition: dohodnina is a tax on the income of natural persons. The retrieved chunks came from the correct ZDoh-2 document, but they were earlier preamble chunks rather than the exact chunk containing the definition. The model consequently answered that the exact information was not present in the retrieved context. This is a desirable behavior from a grounding perspective, but it exposes a retrieval failure.

5.4 Example: Failure on ZDDPO-2 Tax Rate

For the question about the general corporate income tax rate, the expected source was `ZAK04687_zddpo-2_NPB23` Article 60. The retrieved chunks instead came from the ZDDPO-2 rulebook and ZDavP-2. The generated answer correctly avoided inventing the missing rate, but it could not answer the question. This is a clear case where retrieval, not generation, is the limiting component.

5.5 Why the Chunks Differ from the Expected Chunks

For each evaluation item, the expected chunk IDs were derived from the current chunking configuration by locating the statutory article or phrase that contains the reference answer. The retrieved chunks are not the same as these expected chunks in any of the 20 examples. This does not mean that FAISS is broken; it means that the dense embedding model ranks broader semantic similarity above the specific legal citation needed for the answer. The retrieved passages often share terms such as “davčni zavezanec”, “DDV”, “ZDavP-2”, or “olajšava”, but they refer to a different article or an implementing rule. The problem is therefore retrieval precision at the legal-provision level.

5.6 Systematic Retrieval Pattern

Several errors show that semantically related procedural or rulebook documents are often ranked above the exact statutory article. This is especially visible for ZDavP-2 and ZDDPO-2 questions. The embedding model captures broad tax relatedness, but it does not reliably distinguish:

- primary law from implementing rules,
- definitions from procedural mentions,
- article-specific facts from nearby but different provisions.

6 Limitations of the Current Evaluation

The evaluation is useful as a diagnostic baseline, but it has several limitations. First, expected chunk IDs are tied to the current chunking configuration. If chunk size, overlap, or file ordering changes, exact chunk-hit metrics must be refreshed. Second, phrase matching is strict. It is useful for checking grounded context, but it does not capture semantically correct paraphrases. Third, generation quality is not scored automatically. A future version should add manual answer grades or an LLM-as-judge protocol with strict source-grounding checks.

7 Recommendations

The next iteration should focus on retrieval before changing the generator.

1. **Article-aware chunking.** Split legal documents by article headings such as *3. člen*, *41. člen*, and *60. člen*, instead of using only fixed character windows.
2. **Metadata-aware retrieval.** Add metadata fields for law ID, article number, article title, and document type. Use these fields to filter or boost primary statutes over rulebooks when the question names a statute.
3. **Hybrid retrieval.** Combine dense retrieval with BM25 or lexical matching. Many legal questions contain exact anchors such as “41. člen”, “stopnja DDV”, or “ZDDPO-2”, which lexical retrieval should capture.
4. **Reranking.** Retrieve a larger candidate pool, for example top 20, then rerank with a cross-encoder or a stronger local reranker before passing the final top three chunks to the LLM.
5. **Evaluation expansion.** Keep the current 20-question diagnostic set, but add answer-level grading and separate retrieval-only runs for top-k values 3, 5, 10, and 20.
6. **Use explicit citations in questions.** When a question names a law such as ZDDPO-2, use this as a retrieval filter or source boost so that ZDavP-2 and rulebook chunks do not outrank the primary law.
7. **Add chunk-neighborhood expansion.** If a high-scoring chunk is near an expected article boundary, include neighboring chunks before generation. This may recover definitions split across chunk boundaries.

8 Conclusion

The current RAG assistant is operational on HPC: it can build the index, load the local LLM, retrieve context, and generate cited answers. The evaluation shows that the main bottleneck is retrieval precision. With source hit@3 at 40.0%, exact chunk hit@3 at 0.0%, and answer correctness averaging only 0.20/2.00, the system is not yet reliable for factual legal answers. The generator is not the primary engineering failure: it produced an answer for every question and often remained faithful to the retrieved context. The issue is that the retrieved context is frequently the wrong legal provision. The most valuable next step is therefore to improve retrieval through article-aware chunking, metadata extraction, hybrid search, source filtering, chunk-neighborhood expansion, and reranking.